MLR – Practical test 1: to nejdůležitější na 2 A4

25 min · 5 bodů · jen papír, bez poznámek a kalkulačky · zadání anglicky · kód se jen čte · indexy od nuly · zlomky stačí

Co čekat: 5 nezávislých úloh, ~5 min na každou. V obou ukázkách: **matice ručně** (1,25 b.), **oprava kódu**, **interpretace výsledků**, **hlasování nebo výběr příznaků**, **agenti** (0,75 b.).

Plné body: vždy napiš mezikrok a jmenovatel. „Explain" = věta s *because*. U opravy kódu napiš opravený řádek. Začni rychlými body (agenti, predikce), pak výpočty. *Modře* = věta, kterou můžeš použít.

1 · Čtení kódu: nejdřív tvar, pak čísla

všude

Řádek = vzorek, sloupec = příznak. `X` má tvar `(n, d)`; vektor má tvar `(n,)`.

`A @ B`	maticový součin: `(a,b) @ (b,c)` → `(a,c)`. Vnitřní rozměry nesedí = *dimensional error*. `*` násobí po prvcích.
`X.T @ X`	`(d,d)`; `X.T @ y` → `(d,)`; `X @ w` → `(n,)` = jedna predikce na vzorek; `X @ X.T` → `(n,n)`
`axis`	**osa, která zmizí.** `axis=1`: jedno číslo na řádek; `axis=0`: jedno číslo na sloupec
`X.shape[0]`, `[1]`	počet řádků, počet sloupců. `np.eye(k)` = jednotková `k×k`
`column_stack`	skládá vektory jako **sloupce**. `range(3)` = 0, 1, 2; `t**0` = samé jedničky
`(p > 0.5).astype(int)`	práh na 1/0. Ostře: **přesně 0,5 dá 0**
`np.argmax(s)`	**pozice** největšího prvku (při shodě první), ne hodnota
`np.argsort(d)[:k]`	indexy k nejmenších hodnot (k nejbližších sousedů)
`l.pop(i)` / `l.remove(x)`	odebere podle **pozice** a vrátí prvek / odebere podle **hodnoty**
`np.mean(y == p)`	accuracy. `np.sum((y==1) & (p==1))` = TP
`np.unique(v, return_counts=True)`	různé hodnoty a jejich počty (hlasování v k-NN)

2 · Lineární regrese ručně

A1 · 1,25 b.

Princip: hledáme váhy s nejmenším součtem čtverců chyb. Data zapíšeme do matice a váhy dá vzorec. Pro body (−1, 0), (0, 0), (1, 6) a sloupce [1, x]:

$$X = \begin{bmatrix} 1 & -1 \\ 1 & 0 \\ 1 & 1 \end{bmatrix} \qquad \mathbf{y} = \begin{bmatrix} 0 \\ 0 \\ 6 \end{bmatrix} \qquad \mathbf{w} = (X^\top X)^{-1} X^\top \mathbf{y}$$

The column of ones multiplies the intercept b, so b is learned as an ordinary weight.

Když chtějí výpočet:

$$X^\top X = \begin{bmatrix} 3 & 0 \\ 0 & 2 \end{bmatrix} \qquad X^\top \mathbf{y} = \begin{bmatrix} 6 \\ 6 \end{bmatrix} \qquad \mathbf{w} = \begin{bmatrix} 2 \\ 3 \end{bmatrix}$$

Inverze 2×2 (ta se musí umět z hlavy):

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix}^{-1} = \frac{1}{ad - bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$

Predikce: $\hat{y}(2) = b + w \cdot 2 = 2 + 3 \cdot 2 = 8$.

Pozor: „Leave unevaluated" = jen dosaď matice do vzorce, nepočítej. Pořadí sloupců ber ze zadání: u [x, 1] je $\mathbf{w} = [w, b]^\top$ a jedničky jsou vpravo.

3 · Polynomiální příznaky

B1 · 1,25 b.

```
t = np.array([-1., 0., 1.])
# sloupce: 1, t, t²
X = np.column_stack([t ** j for j in range(3)])
y_pred = X @ np.array([1., 2., 1.])
```

$$X = \begin{bmatrix} 1 & -1 & 1 \\ 1 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix} \qquad \texttt{y_pred} = \begin{bmatrix} 0 \\ 1 \\ 4 \end{bmatrix} \qquad \hat{y} = 1 + 2t + t^2$$

The prediction is a weighted sum of fixed features 1, t, t^2; only the features are nonlinear in t, so the model is linear in its weights and the same formula fits them.

Bez konstanty (`range(1, 3)`) model v t = 0 vždy predikuje 0.

4 · Ridge regrese

B2 · 1 b.

Princip: kromě chyby se trestají i velké váhy. Model je pak klidnější a méně se přeučí.

$$J(\mathbf{w}) = \|X\mathbf{w} - \mathbf{y}\|_2^2 + \lambda \|\mathbf{w}\|_2^2$$

$$\mathbf{w} = (X^\top X + \lambda I)^{-1} X^\top \mathbf{y}$$

- **1. člen** měří chybu na trénovacích datech, **2. člen** trestá velké váhy.
- **↑λ:** menší váhy, hladší křivka, trénovací chyba roste. Příliš velké λ = podučení (váhy → 0). λ = 0 je obyčejná regrese.

Chyba z ukázky: `np.eye(X.shape[0])` je 3×3, ale `X.T @ X` je 2×2, takže nejdou sečíst. Oprava: `np.eye(X.shape[1])`, jedna diagonála na váhu.

Duální tvar je jiný: `LA.inv(X @ X.T + lam*np.eye(X.shape[0])) @ y` dá α a $\mathbf{w} = X^\top \boldsymbol{\alpha}$. Tady je `shape[0]` **správně**, protože `X @ X.T` je `(n,n)`. Jádrový trik: $XX^\top$ se nahradí maticí K, pak $\hat{\mathbf{y}} = K_{\text{new}}\boldsymbol{\alpha}$.

Ruční výpočet: stejná data jako v 2, λ = 1, trestají se obě váhy:

$$X^\top X + I = \begin{bmatrix} 4 & 0 \\ 0 & 3 \end{bmatrix} \qquad \mathbf{w} = \begin{bmatrix} 6/4 \\ 6/3 \end{bmatrix} = \begin{bmatrix} 3/2 \\ 2 \end{bmatrix}$$

Váhy se oproti [2, 3] zmenšily. Trest jen na sklon: místo λI se přičte diag(0, λ).

5 · Podučení a přeučení

A4 · 1 b.

Stav	Trénovací chyba	Testovací chyba	Křivka
podučení (st. 1)	vysoká 0,227	vysoká 0,283	nestíhá trend
dobře (st. 3)	nízká 0,051	nejnižší 0,085	sleduje trend
přeučení (st. 15)	nejnižší 0,040	obří 10,39	kmitá, ulétá

Vždy cituj **křivku i čísla z tabulky**. Stupeň, λ, k i příznaky se vybírají na validaci:

Choose the degree on a validation split or by cross-validation; use the test set only once at the end, so it stays independent.

6 · k-NN a gradientní sestup

cvičení 1–2

- **k-NN:** většina mezi k nejbližšími. Ručně stačí d^2 bez odmocniny, pořadí se nezmění. Malé k = citlivé na šum, velké k = rozmazané hranice. Bod (3, 3): d^2 k (2, 3) je 1, k (4, 4) je 2.
- **Sestup:** $w_{t+1} = w_t - \eta f'(w_t)$. Pro $f = (w-3)^2$, $w_0 = 0$, η = 0,25: $w_1 = 1{,}5$, $w_2 = 2{,}25$. Velké η kmitá nebo diverguje, malé je pomalé.

7 · Matice záměn a metriky

B3 · 1 b.

	Pred. 0	Pred. 1
Actual 0	TN = 6	FP = 2
Actual 1	FN = 1	TP = 3

„missed“ = FN = 1
„flagged“ = FP = 2
Nejdřív čti popisky!

$$\text{accuracy} = \frac{TP+TN}{\text{vše}} = \frac{3+6}{12} = \frac{3}{4}$$

$$\text{recall} = \frac{TP}{TP+FN} = \frac{3}{3+1} = \frac{3}{4}$$

$$\text{precision} = \frac{TP}{TP+FP} = \frac{3}{3+2} = \frac{3}{5}$$

$$\text{specificity} = \frac{TN}{TN+FP} = \frac{6}{6+2} = \frac{3}{4}$$

$$F_1 = \frac{2TP}{2TP+FP+FN} = \frac{6}{9} = \frac{2}{3}$$

- **Recall:** *the share of truly faulty items that the model catches (3 of 4).*
- **Precision:** *of the items flagged as faulty, the share that really are faulty.*
- Screening: důležitější recall (přehlédnutí je horší než falešný poplach).
- Nevyvážené třídy: accuracy klame, proto recall, F1, balanced accuracy.
- Regrese: $\text{MSE} = \frac{1}{n}\sum(\hat{y}_i - y_i)^2$, $\text{MAE} = \frac{1}{n}\sum|\hat{y}_i - y_i|$.

> **Pozor:** matice může být otočená (řádky = predikce). Vždy napiš čísla ve jmenovateli.

8 · Předzpracování a únik dat

A2 · 1 b.

Pravidlo: vše, co se učí z dat (μ, σ, min, max, průměr na doplnění), se učí **jen z train**. **Validace** vybírá, **test** se použije jednou na konci.

```
X_train = scaler.fit_transform(train)
X_valid = scaler.transform(valid)     # ne fit_transform!
```

Refitting on validation scales it with a different mean and standard deviation, so the two sets are in different units and k-NN distances are distorted. The mean and standard deviation must come from the training data only.

$$z = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}} = \frac{4-1}{1} = 3$$

Proč škálovat: ve vzdálenosti jinak přehluší vše příznak s velkými čísly. Stromy škálování nepotřebují.

Další chyby v kódu („red flags“):

- scaler nebo průměr na doplnění spočítaný **i z dalších dat**: před rozdělením nebo z `concat([train, valid])`
- výběr k / λ podle testu, `max(scores)` z testu jako výsledek
- accuracy na trénovacích datech (1-NN má skoro vždy 100 %)
- `default_rng(0)` uvnitř smyčky: všechny stromy dostanou stejný vzorek. Generátor vytvoř jednou před smyčkou.

9 · Výběr příznaků

B4 · 1 b.

Dopředný výběr: přidávej po jednom příznak, který nejvíc zvýší validační skóre. Skonči, když se ostře nezlepší.

```
selected = [0];  remaining = [1, 3, 4]
best_score = 0.75
scores = [0.80, 0.90, 0.85]
best_position = int(np.argmax(scores))     # 1
feature = remaining.pop(best_position)     # 3
selected.append(feature)
best_score = scores[best_position]         # 0.90
# selected = [0, 3], remaining = [1, 4]
```

- **Pozice ≠ příznak:** pozice 1 je místo v seznamu, příznak `remaining[1] = 3` je sloupec dat.
- **„Strictly improves“:** 0,90 není > 0,90, takže nepřijmout a skončit.
- **Filtr** (korelace, vzájemná informace) hodnotí každý příznak zvlášť bez modelu. **Wrapper** (dopředný výběr) měří podmnožiny výkonem modelu.
- **PCA** nevybírá, ale skládá nové příznaky ve směrech největšího rozptylu; nepoužívá labely.

10 · Stromy a bagging

A5 · 1 b.

$$V = \begin{bmatrix} 0 & 1 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & 1 \\ 1 & 1 & 1 \end{bmatrix} \qquad \mathbf{y} = \begin{bmatrix} 0 \\ 1 \\ 1 \\ 1 \end{bmatrix}$$

`(V.mean(axis=1) > 0.5).astype(int)`

axis=1 averages over the trees in each row: one value per sample, tedy podíl hlasů pro 1:

$$\left[\tfrac{1}{3}, \tfrac{2}{3}, \tfrac{1}{3}, 1\right] \;\rightarrow\; \hat{\mathbf{y}} = [0, 1, 0, 1] \;\rightarrow\; \text{accuracy} = \tfrac{3}{4}$$

Bootstrap `rng.choice(n, size=n, replace=True)`: *sampling with replacement, so a row can occur twice and some rows are left out (out-of-bag, about 37 %).* Stejné `idx` pro X i y.

- **Bagging:** každý strom na vlastním bootstrapu, pak hlasování. Snižuje rozptyl, ale jen když se stromy liší. **Random forest** navíc v každém uzlu zkouší náhodnou podmnožinu příznaků.
- **Strom:** řezy $x_j \le s$, list predikuje většinovou třídu.
- **Gini:** $G = 2p(1-p)$ pro dvě třídy. Čistý uzel 0, půl na půl 0,5, 4 : 1 → 0,32. Kvalita řezu = vážený průměr dětí.
- **SVM:** ↑C i ↑γ = složitější hranice, riziko přeučení.

11 · AI agenti

A3, B5 · 0,75 b.

Skill	*A saved, reusable recipe (a SKILL.md file with a name, a description of when to use it, and instructions) that the agent loads only when the task needs it.*
Subagent	*A helper the main agent starts for a side task; it works in its own fresh context and returns only a short summary.*
Hook	*A small script the harness runs automatically at an event, e.g. before every file edit; it can block the action.*
`AGENTS.md`	pravidla projektu, čtou se vždy. Pravidlo **žádá**, hook **vynutí**.
MCP	standardní „zástrčka“ pro další nástroje (prohlížeč, dokumentace).
Kontext	vše, co model čte před odpovědí. Plný kontext: stav do `PLAN.md`, pak `/clear`.

Jak vytvořit skill: *Do the task once with the agent and note the steps and the check that worked. Write .claude/skills/<name>/SKILL.md with a name, a clear description and short instructions. Test it in a fresh session and refine it.*

Úkol pro subagenta: *Read the long training log and return only the epoch with the lowest validation loss, its value and any error messages.* Výstup musí být konkrétní a krátký; subagent nevidí konverzaci.

Dobré zadání: Goal · Context · Constraints · Done when. **Důkaz:** *Run the check and show the output.*

Kontrola před odevzdáním

- Sedí orientace matice záměn a pořadí sloupců X podle zadání?
- Jsou tam mezikroky a jmenovatele, ne jen výsledek?
- Je u opravy kódu opravený řádek a u „explain“ důvod (*because*)?
- Pozice, nebo číslo příznaku? `shape[0]`, nebo `shape[1]`?
- Je výběr modelu na validaci a test až na konci?

Slovíčka ze zadání

sample = vzorek · feature = příznak · weight = váha · intercept = absolutní člen · fit = natrénovat · split = část dat · denominator = jmenovatel · missed = přehlédnutý · flagged = označený · trade-off = kompromis · with replacement = s vracením · strictly = ostře · penalize = trestat